

TP — Connexion, inscription et panier d'une boutique en ligne

Technologies : HTML5 / CSS3

1. Contexte

Dans le TP précédent, vous avez créé une petite boutique en ligne avec une page d'accueil, une boutique, une navigation commune, des cartes produits et une feuille de style CSS.

Le projet évolue avec trois nouvelles pages :

```
login.html  
register.html  
panier.html
```

Pour l'instant, aucune donnée n'est enregistrée sur un serveur et aucune authentification réelle n'est mise en place. L'objectif est de construire les interfaces HTML/CSS.

Plus tard, le projet pourra évoluer vers :

```
HTML / CSS → JavaScript → PHP → Base de données
```

2. Objectifs

- créer et organiser des formulaires HTML5 ;
- utiliser différents types de champs et leurs validations ;
- associer `label` et `input` ;
- créer des pages de connexion et d'inscription ;
- créer une page de panier ;
- utiliser Flexbox et une feuille CSS commune ;
- maintenir une navigation cohérente ;
- adapter les pages aux petits écrans ;
- préparer les interfaces à une future dynamisation en JavaScript.

3. Organisation du projet

L'arborescence devient :

```
boutique/  
├─ index.html
```

```
├── boutique.html
├── login.html
├── register.html
├── panier.html
├── script.js
├── css/
│   └── style.css
└── img/
    ├── logo1.png
    ├── carnus.jpg
    ├── mba.png
    ├── mbp.png
    ├── HP.png
    └── Dell.png
```

Question 1

Pourquoi est-il préférable d'utiliser une seule feuille `css/style.css` pour toutes les pages ?

4. Faire évoluer la navigation

La navigation doit permettre d'accéder aux cinq pages :

```
Accueil | Boutique | Connexion | Inscription | Panier
```

Les liens doivent être présents sur toutes les pages.

La page actuellement consultée devra être identifiable grâce à une classe `.active` comme :

```
<a href="login.html" class="active">
  <i class="fa-solid fa-user"></i>
  Connexion
</a>
```

Question 2

Quel est l'intérêt d'utiliser une classe CSS comme `.active` ?

5. Page `register.html`

Créer la page `register.html` en conservant le même `header`, logo, menu, `footer` et fichier CSS.

Le titre sera :

Créer un compte

Formulaire

Créer un formulaire contenant au minimum :

Champ	Type
Prénom	text
Nom	text
Adresse e-mail	email
Mot de passe	password
Confirmation	password
Date de naissance	date
Téléphone	tel

Chaque champ doit posséder un `label` correctement associé.

Exemple :

```
<label for="prenom">Prénom</label>
<input type="text" id="prenom" name="prenom">
```

Validation

Utiliser `required` pour les champs obligatoires.

Le formulaire devra également utiliser plusieurs contraintes HTML5, notamment :

```
<input
  type="password"
  id="password"
  name="password"
  minlength="8"
  required
>
```

Ajouter une contrainte adaptée au téléphone ainsi qu'une longueur minimale pour le nom et le prénom.

Question 3

Quel est le rôle de `required` ?

Question 4

Quelle est la différence entre `required` et `minlength="8"` ?

6. Civilité

Ajouter un choix de civilité :

- ☐ Madame
- ☐ Monsieur
- ☐ Autre

Utiliser des boutons radio avec le même `name` :

```
<input type="radio" name="civilite" value="madame">
<input type="radio" name="civilite" value="monsieur">
<input type="radio" name="civilite" value="autre">
```

Question 5

Pourquoi les boutons radio doivent-ils avoir le même attribut `name` ?

7. Conditions d'utilisation et bouton d'inscription

Ajouter une case à cocher obligatoire :

☐ J'accepte les conditions générales d'utilisation.

Utiliser `required`.

Créer ensuite un bouton :

```
<button type="submit">
  <i class="fa-solid fa-user-plus"></i>
  Créer mon compte
</button>
```

Le bouton devra posséder un effet au survol avec `:hover`.

8. Mise en forme du formulaire

Présenter le formulaire dans une carte centrée.

Utiliser notamment :

```
max-width  
margin: auto  
padding  
border-radius  
box-shadow
```

Question 6

Pourquoi utiliser `max-width: 500px` plutôt qu'une très grande largeur fixe ?

9. Page `login.html`

Créer `login.html` avec le titre :

Connexion

Le formulaire doit contenir :

- une adresse e-mail obligatoire ;
- un mot de passe obligatoire ;
- une case à cocher « Se souvenir de moi » ;
- un bouton « Se connecter » avec une icône.

Utiliser :

```
<input type="email">  
<input type="password">
```

10. Liens entre connexion et inscription

La page de connexion doit proposer un lien vers l'inscription :

Vous n'avez pas encore de compte ?
Créer un compte

Le lien doit pointer vers :

`register.html`

La page d'inscription doit proposer le lien inverse vers :

login.html

Ajouter également :

Mot de passe oublié ?

Le lien peut pour l'instant pointer vers # ou une future page.

Question 7

Pourquoi créer ce lien alors que la fonctionnalité n'existe pas encore ?

11. Page panier.html

Créer panier.html avec le titre :

Mon panier

La page doit représenter un panier contenant plusieurs produits.

Important : le panier est uniquement statique à ce stade. Le bouton « Ajouter au panier » ne doit pas encore modifier cette page.

Cette fonctionnalité sera réalisée plus tard avec JavaScript.

12. Produits du panier

Afficher plusieurs produits avec au minimum :

- une image ;
- le nom ;
- le prix unitaire ;
- la quantité ;
- le prix total ;
- un bouton de suppression.

Exemple :

Produit	Quantité	Prix	Total
---------	----------	------	-------

MacBook Air	1	1890 €	1890 €
MacBook Pro	1	2395 €	2395 €
HP	2	1590 €	3180 €

Pour la suppression :

```
<i class="fa-solid fa-trash"></i>
Supprimer
```

Le bouton n'a pas encore besoin de fonctionner.

13. Quantité

Utiliser un champ numérique :

```
<input
  type="number"
  min="1"
  max="5"
  value="1"
>
```

Question 8

Pourquoi utiliser `type="number"` plutôt que `type="text"` pour la quantité ?

14. Récapitulatif du panier

Afficher un récapitulatif :

Récapitulatif	
Sous-total	7465 €
Livraison	0 €
Total	7465 €
[Passer la commande]	

Les valeurs peuvent être écrites directement dans le HTML.

Aucun calcul automatique n'est demandé. Les calculs seront réalisés plus tard avec JavaScript.

15. Mise en page avec Flexbox

Utiliser Flexbox pour organiser les produits, les quantités, les prix, les boutons et le récapitulatif.

Exemple :

```
.panier-item {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
}
```

Question 9

Quel est l'intérêt de `justify-content: space-between` dans cette situation ?

16. Responsive design

Les trois nouvelles pages doivent être utilisables sur ordinateur, tablette et smartphone.

Sur grand écran, le panier pourra être organisé en deux colonnes :

Liste des produits	Récapitulatif
Produit 1	Sous-total
Produit 2	Livraison
Produit 3	Total

Sur petit écran, les éléments passeront les uns sous les autres.

Liste des produits
Produit 1
Produit 2
Produit 3
Récapitulatif
...
Total

Utiliser notamment :

```
@media (max-width: 768px) {  
  ...  
}
```

Question 10

Pourquoi le panier doit-il passer à une organisation verticale sur un petit écran ?

17. Amélioration du menu

Ajouter des icônes Font Awesome au menu :

Fonction	Icône
Accueil	<code>fa-house</code>
Boutique	<code>fa-store</code>
Connexion	<code>fa-user</code>
Inscription	<code>fa-user-plus</code>
Panier	<code>fa-cart-shopping</code>

Exemple :

```
<a href="panier.html">  
  <i class="fa-solid fa-cart-shopping"></i>  
  Panier  
</a>
```

18. Nombre d'articles du panier

Afficher un badge indiquant le nombre d'articles :

 Panier (3)

Exemple :

```
<span class="cart-count">3</span>
```

Créer une mise en forme CSS permettant d'afficher ce nombre comme un petit badge.

Le nombre peut rester statique. Il ne doit pas encore être calculé automatiquement.

19. Cohérence graphique

Les nouvelles pages doivent reprendre l'identité graphique de `index.html` et `boutique.html`.

Conserver notamment :

- les couleurs ;
- la police ;
- le logo ;
- le header et le footer ;
- la navigation ;
- les boutons ;
- les arrondis ;
- les ombres ;
- les espacements.

Question 11

Pourquoi est-il important de conserver une identité graphique cohérente entre les pages d'un même site ?

20. Tester la validation HTML5

Tester les formulaires avec différentes valeurs.

Test 1

Laisser l'adresse e-mail vide.

Test 2

Entrer :

bonjour

dans le champ e-mail.

Test 3

Entrer un mot de passe trop court.

Test 4

Ne pas accepter les conditions d'utilisation.

Pour chaque test, observer le comportement du navigateur.

Question 12

Qui effectue cette première validation ?

- HTML ;
- CSS ;
- JavaScript ;
- serveur.

Justifiez votre réponse.

21. Limites de cette version

À la fin du TP, les pages doivent être correctement réalisées visuellement, mais l'application reste statique.

Elle ne permet pas encore de :

- sauvegarder un compte ;
- vérifier une connexion ;
- modifier réellement le panier ;
- supprimer un produit ;
- recalculer le total ;
- modifier automatiquement le nombre d'articles.

Ces fonctionnalités seront ajoutées progressivement avec JavaScript, puis avec PHP et une base de données.

22. Validation du travail

HTML

- ☐ Les cinq pages fonctionnent.
- ☐ Les titres sont corrects.
- ☐ Les images possèdent un **alt**.
- ☐ Les formulaires utilisent des **label**.
- ☐ Les **id** et **for** sont correctement associés.
- ☐ Les champs obligatoires utilisent **required**.
- ☐ Les types HTML5 sont correctement utilisés.

Navigation

- ☐ Accueil est accessible.
- ☐ Boutique est accessible.
- ☐ Connexion est accessible.
- ☐ Inscription est accessible.
- ☐ Panier est accessible.

- ☐ La page active est identifiable.
- ☐ Les liens connexion/inscription fonctionnent.

Inscription

- ☐ Prénom et nom.
- ☐ Civilité.
- ☐ E-mail.
- ☐ Téléphone.
- ☐ Date de naissance.
- ☐ Mot de passe et confirmation.
- ☐ Conditions d'utilisation.
- ☐ Bouton d'inscription.

Connexion

- ☐ E-mail.
- ☐ Mot de passe.
- ☐ « Se souvenir de moi ».
- ☐ Bouton de connexion.
- ☐ Lien vers l'inscription.
- ☐ Lien « Mot de passe oublié ».

Panier

- ☐ Plusieurs produits.
- ☐ Images et noms.
- ☐ Prix et quantités.
- ☐ Champs `type="number"`.
- ☐ Boutons de suppression.
- ☐ Sous-total.
- ☐ Livraison.
- ☐ Total.
- ☐ Bouton « Passer la commande ».
- ☐ Badge du nombre d'articles.

CSS

- ☐ Toutes les pages utilisent `style.css`.
- ☐ L'identité graphique est cohérente.
- ☐ Les formulaires sont correctement centrés.
- ☐ Les champs sont espacés.
- ☐ Les boutons possèdent un effet `:hover`.
- ☐ Flexbox est utilisé lorsque cela est pertinent.
- ☐ Les pages sont responsive.
- ☐ Les pages sont utilisables sur smartphone.

23. Livrable

Rendre l'ensemble du projet :

```
boutique/
├── index.html
├── boutique.html
├── login.html
├── register.html
├── panier.html
├── script.js
├── css/
│   └── style.css
├── img/
│   ├── logo1.png
│   ├── carnus.jpg
│   ├── mba.png
│   ├── mbp.png
│   ├── HP.png
│   └── Dell.png
```

Le code doit être :

- propre ;
- correctement indenté ;
- organisé ;
- compréhensible ;
- cohérent entre les différentes pages.

24. Barème indicatif — /20

Critère	Points
Organisation du projet	1
Navigation entre les pages	2
<code>register.html</code> : structure du formulaire	3
Validation HTML5	2
<code>login.html</code>	2
<code>panier.html</code> : structure	3
Flexbox et mise en page du panier	2
Cohérence graphique / CSS	1
Responsive design	2
Qualité du code	2
Total	20

Bonus

- **+1** : affichage/masquage du mot de passe.
 - **+1** : amélioration graphique du panier ou des formulaires.
 - **+1** : version particulièrement adaptée aux smartphones.
-

25. Conclusion

Dans ce TP, vous avez ajouté :

```
login.html  
register.html  
panier.html
```

Vous avez travaillé les formulaires HTML5, leur validation, la navigation, Flexbox, le responsive design et la cohérence graphique.

À ce stade, l'application reste essentiellement statique.
